single scalar c that best matches normalized activations in least squares

$$c^\star \in \arg\min_c \mathbb{E}\left[\left\|\frac{h}{r(h)}D_\gamma - c\,hD_\gamma\right\|_2^2\right] = \frac{\mathbb{E}\big[\|hD_\gamma\|_2^2/r(h)\big]}{\mathbb{E}[\|hD_\gamma\|_2^2]}\,. \tag{4}$$

This matches the two inputs in energy under the layer's current scaling, not in raw coordinates. The derivation is given in Appendix A.5.

3.5. Fixed-target scale anchoring loss

In addition to the tapering mechanism, we use an auxiliary loss that anchors the scale of the pre-logit residual stream. When a final normalization is present, this stabilizes the residual-stream scale during tapering by reducing activation variance. When the final normalization is tapered away, it also provides an explicit alternative to the final norm's implicit scale anchoring. Let token vectors before the final normalization, rows of $H^{(L,0)}$, be denoted using h_t. Define a per-token scale statistic

$$s(h) = \begin{cases} r(h) = \sqrt{\|h\|_2^2/d + \varepsilon} & \text{for RMSNorm,} \\ \sigma_h = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(h_i - \mu_h)^2 + \varepsilon} & \text{for LayerNorm.} \end{cases}$$

The auxiliary loss penalizes deviations of token scales from a single scalar target

$$\mathcal{L}_{\text{aux}} \;=\; \lambda\,\mathbb{E}_{b,t}(s(h_{b,t}) - s_{\text{tgt}})^2\,, \tag{5}$$

where (b,t) index batch elements and sequence positions, and the expectation is taken over non-padding tokens. While $g = 1$ (gate warmup) we track an EMA of the batch mean $\hat{s}_k = \mathbb{E}_{b,t}[s(h_{b,t})]$, and at taper start we freeze s_{tgt} to the bias-corrected EMA value. After taper start, s_{tgt} remains constant for the remainder of training. We train with the combined objective $\mathcal{L} = \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{aux}}$, where $\mathcal{L}_{\text{CE}}$ is token-level cross-entropy.

4. Scale anchoring

This section isolates the mechanism that explains the training behavior we observe when removing normalization. It is control of the scale of the pre-logit representation. When per-token normalization is removed, the scale of the last hidden state becomes a degree of freedom that can drift during training. A final normalization layer anchors it implicitly. Our fixed-target scale loss anchors it explicitly.

We analyze a single token vector $h \in \mathbb{R}^{1\times d}$ that is input to the output projection. The logits are

$$z = \text{Norm}_{\text{final}}(h)\,W_{\text{out}} \in \mathbb{R}^{1\times V}.$$

Only the radial component $\langle \nabla_h \ell(z,y), h\rangle$ can change $\|h\|_2$ to first order. The propositions below characterize this radial component for three cases that correspond to the design choices in our method. Proofs can be found in Appendix A.

Proposition 4.1 (Final normalization removes radial gradient)**.** *Let the final map* $\text{Norm}_{\text{final}}$ *before the output projection be* 0*-homogeneous and differentiable almost everywhere. This includes the idealized RMSNorm and LayerNorm maps with* $\varepsilon = 0$*. For logits* $z = \text{Norm}_{\text{final}}(h)\,W_{\text{out}}$ *and any differentiable loss* $\ell(z,y)$*,*

$$\langle \nabla_h\,\ell(z,y),\, h\rangle = 0 \quad (h \neq 0).$$

Proposition 4.1 shows that normalization removes the radial gradient at the output. In this setting the loss cannot be reduced by simply rescaling the last hidden state.

Proposition 4.2 (Without the final norm, cross-entropy pushes norms up)**.** *If* $z = h\,W_{\text{out}}$ *and* ℓ *is multiclass cross-entropy, then whenever the margin* $m = z_y - \max_{j\neq y} z_j > 0$*,*

$$\langle \nabla_h \ell(z,y), h\rangle = \langle \nabla_z \ell, z\rangle \leq -(1 - \text{softmax}(z)_y)m < 0,$$

so a small gradient step along $-\nabla_h \ell$ *increases* $\|h\|_2^2$ *to first order.*

Proposition 4.2 captures a simple mechanism for unbounded scale growth when the final normalization is removed. Once the correct class has positive margin, scaling logits along their current direction reduces cross-entropy. We refer to this tendency toward increasing confidence through logit magnitude as logit chasing.

Proposition 4.3 (Fixed-target scale loss provides a radial restoring force)**.** *Let* $r(h) = \sqrt{\|h\|_2^2/d + \varepsilon}$ *and define the fixed-target auxiliary loss on a token vector* h *by*

$$\mathcal{L}_{\text{aux}}(h) = \lambda\,(r(h) - s_{\text{tgt}})^2, \qquad \lambda > 0.$$

Then

$$\nabla_h \mathcal{L}_{\text{aux}}(h) = \frac{2\lambda\,(r(h) - s_{\text{tgt}})}{d\,r(h)}\,h,$$
$$\langle \nabla_h \mathcal{L}_{\text{aux}}(h),\, h\rangle = \frac{2\lambda\,(r(h) - s_{\text{tgt}})}{d\,r(h)}\,\|h\|_2^2.$$

In particular, if $r(h) > s_{\text{tgt}}$ *then a small gradient step on* $\mathcal{L}_{\text{aux}}$ *decreases* $\|h\|_2$ *to first order, while if* $r(h) < s_{\text{tgt}}$ *it increases* $\|h\|_2$*.*

Proposition 4.3 shows that the auxiliary loss injects an explicit restoring force on scale. For LayerNorm with $s(h) = \sigma_h$, the same conclusion holds with the restoring direction given by the mean-centered vector. The derivative formulas are given in Appendix A.4.

Taken together, Propositions 4.1–4.3 motivate our default training choices. Keeping the final normalization provides an implicit anchor that removes the radial component at the output. If we also taper away the final normalization, the fixed-target scale loss provides an explicit alternative that counteracts the radial push from cross-entropy.

5. Experiments

We evaluate TaperNorm in three settings. First, we pre-train pre-norm Transformers on TinyStories to test whether TaperNorm can match training with RMSNorm. Second, we run a microbenchmark to quantify inference gains from folding the resulting sample-independent scalings into adjacent projections. Third, we fine-tune GPT-2 models using the LayerNorm variant and compare to prior LayerNorm removal results.

5.1. Pre-training

We test pre-training performance on TinyStories with causal, pre-norm Transformers (depth 8, 16 attention heads at all widths, SwiGLU MLPs, RoPE) while varying width to obtain models at a range of scales from 1 to 30 million parameters. We repeat each configuration with 6 random seeds and report mean and standard deviation. Evaluation of models containing TaperNorm layers is performed with the gate set to 0, i.e. no per-token statistics used. Shaded regions in the plots indicate the 2.5–97.5 percentile range across seeds for each configuration.

All pre-training runs use a SentencePiece tokenizer trained on TinyStories (vocabulary size 10k) and a context length of 512 tokens. We use AdamW with peak learning rate 3×10^{-4}, $\beta = (0.9, 0.95)$, 5% linear warmup, and cosine decay to zero. We use no dropout or weight decay in these experiments.

Our default TaperNorm setting includes the fixed-target scale anchoring loss from Section 3.5. For ablations we also report runs without it. Among the taper variants, Internal-Taper (+aux) with the final RMSNorm retained is the most stable and reliable across seeds, and we use it as the default configuration in our main comparisons. We additionally report All-Taper (+aux) to quantify the fully norm-free setting enabled by explicit scale anchoring. For the 9M All-Taper (+aux) setting, one of the 6 seeds produced an unusually high validation loss, and we report statistics over the remaining 5 seeds (Table 2, †). The settings investigated are summarized in Table 1.

For all models containing TaperNorm layers we follow this gate schedule. During learning-rate warmup the gate is fixed at $g{=}1$. At the warmup boundary we compute $c^\star$ from γ-weighted EMAs with bias correction, copy $\gamma \to \tilde{\gamma}$, and then freeze $c^\star$. Post-warmup, g is cosine-decayed to 0. All other training details between same-size models are identical.

Table 1. Settings investigated for TinyStories pre-training. "Internal taper" and "Final taper" mean the corresponding normalization layers are replaced by TaperNorm.

Variant	Internal taper	Final taper	Scale loss $\mathcal{L}_{\text{aux}}$
Baseline (all RMSNorm)	×	×	×
Internal-Taper	✓	×	×
Internal-Taper (+aux)	✓	×	✓
Final-Taper	×	✓	×
All-Taper	✓	✓	×
All-Taper (+aux)	✓	✓	✓

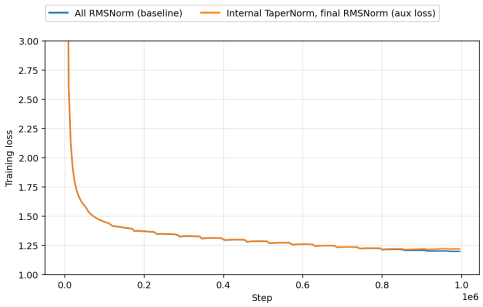

Figure 2. Training loss vs. step for the Baseline and Internal-Taper (+aux).

Figure 2 shows that with the final normalization kept in place, training loss trajectories for RMSNorm and TaperNorm are nearly indistinguishable. The only differences come near the end of training from the complete removal of per-token statistics in the internal layers. In Table 2 we report final validation losses for the Baseline, Internal-Taper (+aux), and All-Taper (+aux). Both taper variants track the baseline closely across scales: Internal-Taper (+aux) is within about 0.7–1.5% relative loss, while All-Taper (+aux) is within about 0.8–1.8%.

Figures 3 and 4 show that training dynamics are governed by scale anchoring at the pre-logit representation. Without an explicit anchor, removing the final normalization layer drastically alters optimization geometry with cross-entropy inducing rapid growth in logit magnitudes (logit chasing), consistent with Proposition 4.2. This effect is reflected in both activation-scale statistics and gradient magnitudes across parameter groups. When we enable the fixed-target scale loss from Section 3.5, the All-Taper model exhibits gradient magnitudes similar to those of the Internal-Taper model that keeps the final RMSNorm. This shows that explicit scale anchoring largely aligns the two.

5.2. Inference efficiency benchmarks

A key practical benefit of tapering normalization is that, once $g \to 0$, each TaperNorm becomes a sample-

Table 2. Validation loss for Baseline, Internal-Taper (+aux), and All-Taper (+aux). Percentages in parentheses show the relative gap vs. the Baseline. † For 9M All-Taper (+aux), we report mean±std over 5 seeds after excluding one outlier run with unusually high validation loss.

Model size	Baseline	Internal-Taper (+aux)	All-Taper (+aux)
1M	2.1538±0.0025	2.1852±0.0040 (+1.46%)	2.1930±0.0035 (+1.82%)
3M	1.7561±0.0010	1.7828±0.0029 (+1.52%)	1.7874±0.0028 (+1.78%)
9M	1.4620±0.0022	1.4837±0.0016 (+1.48%)	1.4837±0.0023† (+1.48%)
30M	1.2768±0.0018	1.2859±0.0010 (+0.71%)	1.2868±0.0041 (+0.78%)

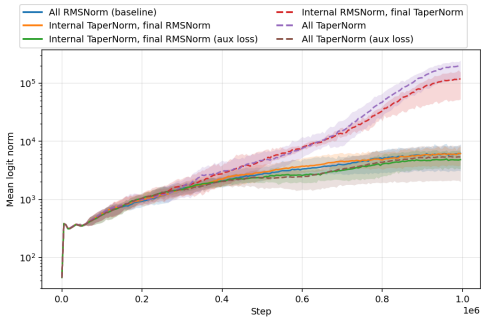

Figure 3. Mean logit ℓ_2 norm vs. step. Without a scale anchor, models with the final norm removed (dashed) can exhibit logit chasing, consistent with Proposition 4.2. With the fixed-target scale loss, logit growth is strongly suppressed in the All-Taper setting.

independent map and can be removed from the inference graph by folding its effect into adjacent linear projections. We quantify the resulting speedup on the TinyStories pretrained Internal-Taper (aux) d=512 model ($\approx$ 30M parameters). We focus on the Internal-Taper model since this setting is more robust than All-Taper while providing near identical computational savings.

We benchmark on a single NVIDIA H100 80GB (CUDA 12.1, PyTorch 2.4.1+cu121) using bf16. We enable TF32 matmul where applicable. We time forward passes in last-token logits mode (only the final position's logits are materialized), without KV caching, using token blocks sampled from the TinyStories validation split. Latency is averaged over 50 timed iterations after 10 warmup iterations using CUDA event timing, and throughput is reported in thousands of tokens per second (k tok/s).

We compare the following settings.

- **Baseline** (RMSNorm).
- **Internal-Taper (unfused)** where TaperNorm layers are exported to a fixed elementwise scaling but left as explicit operations.
- **Internal-Taper (fused)** where the internal scalings are folded into the attention QKV and MLP input projections, allowing normalization modules to be replaced by identity.

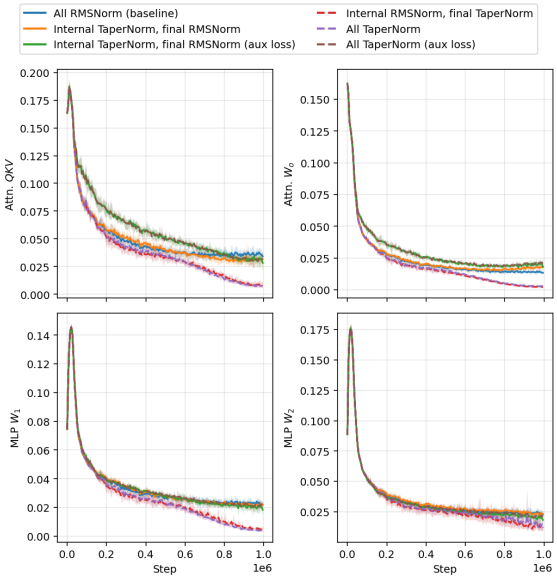

Figure 4. Average gradient norms across all Transformer blocks by weight type. Without explicit scale anchoring, gradients cluster primarily by presence vs. absence of the final normalization. With the fixed-target scale loss enabled, the gradient-magnitude gap between Internal-Taper and All-Taper largely disappears.

Table 3. Inference throughput (k tok/s, higher is better) in last-token logits mode on an NVIDIA H100 80GB (bf16, PyTorch 2.4.1+cu121, CUDA 12.1). Parentheses show speedup relative to the Baseline RMSNorm model.

Batch B	Seq. T	Baseline	Internal-Taper (unfused)	Internal-Taper (fused)
1	128	40.7	45.9 (1.13×)	48.7 (1.20×)
1	256	79.3	90.8 (1.14×)	96.8 (1.22×)
1	512	163.2	181.7 (1.11×)	195.5 (1.20×)
4	128	149.3	169.3 (1.13×)	180.4 (1.21×)
4	256	290.5	323.6 (1.11×)	344.9 (1.19×)
4	512	546.3	588.1 (1.08×)	615.9 (1.13×)

Table 3 reports inference throughput. Across the settings, the unfused TaperNorm model yields a consistent 1.08–1.15× throughput improvement over the RMSNorm baseline, while fully fusing the internal scaling into adjacent projections yields 1.13–1.22× throughput improvement. Gains are larger when per-token normalization is a larger fraction of the forward-pass cost, for example shorter sequences, while for larger T the attention and MLP compute become more significant.